

```
#!/usr/bin/env python3
"""
Technology Search Dictionary and API Interface
Comprehensive search engine for all technologies, systems, applications,
and components
Provides intelligent search, filtering, and recommendation capabilities
"""

import json
import sqlite3
import datetime
import re
from typing import Dict, List, Any, Optional, Tuple
from dataclasses import dataclass, asdict
from enum import Enum
import difflib
from collections import defaultdict
import math

class SearchType(Enum):
    EXACT = "exact"
    FUZZY = "fuzzy"
    SEMANTIC = "semantic"
    CATEGORY = "category"
    TAG = "tag"
    COMPATIBILITY = "compatibility"

class SortBy(Enum):
    RELEVANCE = "relevance"
    POPULARITY = "popularity"
    RECENCY = "recency"
    COMPLEXITY = "complexity"
    MATURITY = "maturity"

@dataclass
class SearchResult:
    """Represents a search result with relevance scoring"""
    technology_id: str
    name: str
    category: str
    description: str
    relevance_score: float
    match_type: str
    matched_fields: List[str]
    tags: List[str]
    github_stars: int
    maturity_level: str
    last_updated: str

@dataclass
class SearchFilter:
    """Represents search filters"""
    categories: Optional[List[str]] = None
    maturity_levels: Optional[List[str]] = None
```

```

license_types: Optional[List[str]] = None
min_github_stars: Optional[int] = None
max_complexity: Optional[int] = None
platforms: Optional[List[str]] = None
programming_languages: Optional[List[str]] = None
date_range: Optional[Tuple[str, str]] = None

class TechnologySearchDictionary:
    """Advanced search engine for technology database"""

    def __init__(self, db_path: str = "technology_database.db"):
        self.db_path = db_path
        self.search_index = {}
        self.category_index = {}
        self.tag_index = {}
        self.compatibility_index = {}
        self.build_search_indices()

    def build_search_indices(self):
        """Build search indices for fast lookup"""
        print("Building search indices...")

        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()

        cursor.execute('''
            SELECT id, name, category, description, tags,
compatible_with,
            programming_languages, platforms, github_stars,
maturity_level, last_updated
            FROM technologies
        ''')

        technologies = cursor.fetchall()

        for tech in technologies:
            tech_id, name, category, description, tags, compatible_with,
prog_langs, platforms, stars, maturity, updated = tech

            # Parse JSON fields
            tags_list = json.loads(tags or '[]')
            compatible_list = json.loads(compatible_with or '[]')
            prog_langs_list = json.loads(prog_langs or '[]')
            platforms_list = json.loads(platforms or '[]')

            # Build main search index
            search_text = f"{name} {description} {'
'.join(tags_list)}".lower()
            self.search_index[tech_id] = {
                'name': name,
                'category': category,
                'description': description,
                'search_text': search_text,
                'tags': tags_list,

```

```

        'compatible_with': compatible_list,
        'programming_languages': prog_langs_list,
        'platforms': platforms_list,
        'github_stars': stars or 0,
        'maturity_level': maturity,
        'last_updated': updated
    }

    # Build category index
    if category not in self.category_index:
        self.category_index[category] = []
    self.category_index[category].append(tech_id)

    # Build tag index
    for tag in tags_list:
        if tag not in self.tag_index:
            self.tag_index[tag] = []
        self.tag_index[tag].append(tech_id)

    # Build compatibility index
    for compatible_tech in compatible_list:
        if compatible_tech not in self.compatibility_index:
            self.compatibility_index[compatible_tech] = []
        self.compatibility_index[compatible_tech].append(tech_id)

    conn.close()
    print(f"Search indices built for {len(self.search_index)}
technologies")

    def search(self, query: str, search_type: SearchType =
SearchType.FUZZY,
                filters: Optional[SearchFilter] = None, sort_by: SortBy =
SortBy.RELEVANCE,
                limit: int = 20) -> List[SearchResult]:
        """Main search function with multiple search types and filters"""

        if search_type == SearchType.EXACT:
            results = self._exact_search(query)
        elif search_type == SearchType.FUZZY:
            results = self._fuzzy_search(query)
        elif search_type == SearchType.SEMANTIC:
            results = self._semantic_search(query)
        elif search_type == SearchType.CATEGORY:
            results = self._category_search(query)
        elif search_type == SearchType.TAG:
            results = self._tag_search(query)
        elif search_type == SearchType.COMPATIBILITY:
            results = self._compatibility_search(query)
        else:
            results = self._fuzzy_search(query) # Default to fuzzy

        # Apply filters
        if filters:
            results = self._apply_filters(results, filters)

```

```

        # Sort results
        results = self._sort_results(results, sort_by)

        return results[:limit]

def _exact_search(self, query: str) -> List[SearchResult]:
    """Exact string matching search"""
    results = []
    query_lower = query.lower()

    for tech_id, tech_data in self.search_index.items():
        relevance_score = 0
        matched_fields = []

        # Check exact matches
        if query_lower == tech_data['name'].lower():
            relevance_score = 100
            matched_fields.append('name')
        elif query_lower in tech_data['name'].lower():
            relevance_score = 80
            matched_fields.append('name')
        elif query_lower in tech_data['description'].lower():
            relevance_score = 60
            matched_fields.append('description')
        elif any(query_lower == tag.lower() for tag in
tech_data['tags']):
            relevance_score = 70
            matched_fields.append('tags')

        if relevance_score > 0:
            results.append(SearchResult(
                technology_id=tech_id,
                name=tech_data['name'],
                category=tech_data['category'],
                description=tech_data['description'],
                relevance_score=relevance_score,
                match_type='exact',
                matched_fields=matched_fields,
                tags=tech_data['tags'],
                github_stars=tech_data['github_stars'],
                maturity_level=tech_data['maturity_level'],
                last_updated=tech_data['last_updated']
            ))

    return results

def _fuzzy_search(self, query: str) -> List[SearchResult]:
    """Fuzzy string matching search with similarity scoring"""
    results = []
    query_lower = query.lower()
    query_words = query_lower.split()

    for tech_id, tech_data in self.search_index.items():

```

```

        relevance_score = 0
        matched_fields = []

        # Name similarity
        name_similarity = difflib.SequenceMatcher(None, query_lower,
tech_data['name'].lower()).ratio()
        if name_similarity > 0.6:
            relevance_score += name_similarity * 100
            matched_fields.append('name')

        # Description word matching
        description_words = tech_data['description'].lower().split()
        word_matches = sum(1 for word in query_words if any(word in
desc_word for desc_word in description_words))
        if word_matches > 0:
            description_score = (word_matches / len(query_words)) *
60
            relevance_score += description_score
            matched_fields.append('description')

        # Tag matching
        tag_matches = sum(1 for word in query_words if any(word in
tag.lower() for tag in tech_data['tags']))
        if tag_matches > 0:
            tag_score = (tag_matches / len(query_words)) * 70
            relevance_score += tag_score
            matched_fields.append('tags')

        # Search text full matching
        search_text_score = 0
        for word in query_words:
            if word in tech_data['search_text']:
                search_text_score += 10
        relevance_score += search_text_score

    if relevance_score > 20: # Minimum threshold
        results.append(SearchResult(
            technology_id=tech_id,
            name=tech_data['name'],
            category=tech_data['category'],
            description=tech_data['description'],
            relevance_score=min(relevance_score, 100),
            match_type='fuzzy',
            matched_fields=matched_fields,
            tags=tech_data['tags'],
            github_stars=tech_data['github_stars'],
            maturity_level=tech_data['maturity_level'],
            last_updated=tech_data['last_updated']
        ))

    return results

def _semantic_search(self, query: str) -> List[SearchResult]:
    """Semantic search using keyword relationships and context"""

```

```

results = []
query_lower = query.lower()

# Define semantic relationships
semantic_mappings = {
    'ai': ['artificial intelligence', 'machine learning', 'deep
learning', 'neural network', 'ml', 'ai'],
    'web': ['website', 'frontend', 'backend', 'http', 'html',
'css', 'javascript'],
    'mobile': ['android', 'ios', 'smartphone', 'app', 'mobile'],
    'database': ['db', 'sql', 'nosql', 'data', 'storage',
'persistence'],
    'cloud': ['aws', 'azure', 'gcp', 'serverless', 'saas',
'paas', 'iaas'],
    'blockchain': ['crypto', 'bitcoin', 'ethereum', 'smart
contract', 'defi', 'web3'],
    'security': ['encryption', 'authentication', 'authorization',
'ssl', 'tls', 'cybersecurity']
}

# Expand query with semantic terms
expanded_terms = [query_lower]
for category, terms in semantic_mappings.items():
    if any(term in query_lower for term in terms):
        expanded_terms.extend(terms)

# Search with expanded terms
for tech_id, tech_data in self.search_index.items():
    relevance_score = 0
    matched_fields = []

    for term in expanded_terms:
        # Name matching
        if term in tech_data['name'].lower():
            relevance_score += 30
            matched_fields.append('name')

        # Description matching
        if term in tech_data['description'].lower():
            relevance_score += 20
            matched_fields.append('description')

        # Tag matching
        if any(term in tag.lower() for tag in tech_data['tags']):
            relevance_score += 25
            matched_fields.append('tags')

        # Category matching
        if term in tech_data['category'].lower():
            relevance_score += 35
            matched_fields.append('category')

    if relevance_score > 15:
        results.append(SearchResult(

```

```

        technology_id=tech_id,
        name=tech_data['name'],
        category=tech_data['category'],
        description=tech_data['description'],
        relevance_score=min(relevance_score, 100),
        match_type='semantic',
        matched_fields=list(set(matched_fields)),
        tags=tech_data['tags'],
        github_stars=tech_data['github_stars'],
        maturity_level=tech_data['maturity_level'],
        last_updated=tech_data['last_updated']
    ))

    return results

def _category_search(self, category: str) -> List[SearchResult]:
    """Search by technology category"""
    results = []
    category_lower = category.lower()

    # Find matching categories
    matching_categories = []
    for cat in self.category_index.keys():
        if category_lower in cat.lower() or cat.lower() in
category_lower:
            matching_categories.append(cat)

    for category in matching_categories:
        tech_ids = self.category_index[category]
        for tech_id in tech_ids:
            tech_data = self.search_index[tech_id]
            results.append(SearchResult(
                technology_id=tech_id,
                name=tech_data['name'],
                category=tech_data['category'],
                description=tech_data['description'],
                relevance_score=90,
                match_type='category',
                matched_fields=['category'],
                tags=tech_data['tags'],
                github_stars=tech_data['github_stars'],
                maturity_level=tech_data['maturity_level'],
                last_updated=tech_data['last_updated']
            ))

    return results

def _tag_search(self, tag: str) -> List[SearchResult]:
    """Search by technology tags"""
    results = []
    tag_lower = tag.lower()

    # Find matching tags
    matching_tags = []

```

```

        for existing_tag in self.tag_index.keys():
            if tag_lower in existing_tag.lower() or existing_tag.lower()
in tag_lower:
                matching_tags.append(existing_tag)

    for tag in matching_tags:
        tech_ids = self.tag_index[tag]
        for tech_id in tech_ids:
            tech_data = self.search_index[tech_id]
            results.append(SearchResult(
                technology_id=tech_id,
                name=tech_data['name'],
                category=tech_data['category'],
                description=tech_data['description'],
                relevance_score=85,
                match_type='tag',
                matched_fields=['tags'],
                tags=tech_data['tags'],
                github_stars=tech_data['github_stars'],
                maturity_level=tech_data['maturity_level'],
                last_updated=tech_data['last_updated']
            ))

    return results

def _compatibility_search(self, technology: str) ->
List[SearchResult]:
    """Search for technologies compatible with the given
    technology"""
    results = []
    tech_lower = technology.lower()

    # Find technologies that list this as compatible
    if tech_lower in self.compatibility_index:
        tech_ids = self.compatibility_index[tech_lower]
        for tech_id in tech_ids:
            tech_data = self.search_index[tech_id]
            results.append(SearchResult(
                technology_id=tech_id,
                name=tech_data['name'],
                category=tech_data['category'],
                description=tech_data['description'],
                relevance_score=95,
                match_type='compatibility',
                matched_fields=['compatible_with'],
                tags=tech_data['tags'],
                github_stars=tech_data['github_stars'],
                maturity_level=tech_data['maturity_level'],
                last_updated=tech_data['last_updated']
            ))

    return results

```



```

    def _apply_filters(self, results: List[SearchResult], filters:
SearchFilter) -> List[SearchResult]:
        """Apply search filters to results"""
        filtered_results = []

        for result in results:
            tech_data = self.search_index[result.technology_id]

            # Category filter
            if filters.categories and result.category not in
filters.categories:
                continue

            # Maturity level filter
            if filters.maturity_levels and result.maturity_level not in
filters.maturity_levels:
                continue

            # GitHub stars filter
            if filters.min_github_stars and result.github_stars <
filters.min_github_stars:
                continue

            # Platform filter
            if filters.platforms:
                if not any(platform in tech_data['platforms'] for
platform in filters.platforms):
                    continue

            # Programming language filter
            if filters.programming_languages:
                if not any(lang in tech_data['programming_languages'] for
lang in filters.programming_languages):
                    continue

            filtered_results.append(result)

        return filtered_results

    def _sort_results(self, results: List[SearchResult], sort_by: SortBy)
-> List[SearchResult]:
        """Sort search results by specified criteria"""
        if sort_by == SortBy.RELEVANCE:
            return sorted(results, key=lambda x: x.relevance_score,
reverse=True)
        elif sort_by == SortBy.POPULARITY:
            return sorted(results, key=lambda x: x.github_stars,
reverse=True)
        elif sort_by == SortBy.RECENCY:
            return sorted(results, key=lambda x: x.last_updated,
reverse=True)
        elif sort_by == SortBy.MATURITY:
            maturity_order = {'experimental': 1, 'alpha': 2, 'beta': 3,
'stable': 4, 'mature': 5, 'legacy': 3, 'deprecated': 0}

```

```

        return sorted(results, key=lambda x:
maturity_order.get(x.maturity_level, 3), reverse=True)
    else:
        return results

    def get_suggestions(self, partial_query: str, limit: int = 10) ->
List[str]:
        """Get search suggestions based on partial query"""
        suggestions = set()
        partial_lower = partial_query.lower()

        # Technology names
        for tech_data in self.search_index.values():
            if tech_data['name'].lower().startswith(partial_lower):
                suggestions.add(tech_data['name'])

        # Tags
        for tag in self.tag_index.keys():
            if tag.lower().startswith(partial_lower):
                suggestions.add(tag)

        # Categories
        for category in self.category_index.keys():
            if category.lower().startswith(partial_lower):
                suggestions.add(category.replace('_', ' ').title())

        return sorted(list(suggestions))[:limit]

    def get_popular_searches(self, limit: int = 10) -> List[Dict[str,
Any]]:
        """Get popular search terms based on technology popularity"""
        popular_techs = []

        for tech_id, tech_data in self.search_index.items():
            popularity_score = tech_data['github_stars'] +
len(tech_data['tags']) * 100
            popular_techs.append({
                'name': tech_data['name'],
                'category': tech_data['category'],
                'popularity_score': popularity_score,
                'github_stars': tech_data['github_stars']
            })

        popular_techs.sort(key=lambda x: x['popularity_score'],
reverse=True)
        return popular_techs[:limit]

    def get_trending_technologies(self, days: int = 30, limit: int = 10)
-> List[Dict[str, Any]]:
        """Get trending technologies based on recent updates and
popularity"""
        trending = []
        cutoff_date = datetime.datetime.now() -
datetime.timedelta(days=days)

```

```

        for tech_id, tech_data in self.search_index.items():
            try:
                last_updated =
datetime.datetime.fromisoformat(tech_data['last_updated'].replace('Z',
'+00:00'))

                if last_updated > cutoff_date:
                    trending_score = tech_data['github_stars'] +
(len(tech_data['tags']) * 50)
                    trending.append({
                        'name': tech_data['name'],
                        'category': tech_data['category'],
                        'trending_score': trending_score,
                        'last_updated': tech_data['last_updated'],
                        'github_stars': tech_data['github_stars']
                    })
            except:
                continue # Skip invalid dates

        trending.sort(key=lambda x: x['trending_score'], reverse=True)
        return trending[:limit]

def get_technology_stats(self) -> Dict[str, Any]:
    """Get comprehensive statistics about the technology database"""
    stats = {
        'total_technologies': len(self.search_index),
        'categories': {},
        'maturity_levels': {},
        'top_tags': {},
        'github_stats': {
            'total_stars': 0,
            'avg_stars': 0,
            'max_stars': 0,
            'technologies_with_repos': 0
        }
    }

    # Category distribution
    for category, tech_ids in self.category_index.items():
        stats['categories'][category] = len(tech_ids)

    # Maturity levels
    maturity_counts = defaultdict(int)
    total_stars = 0
    max_stars = 0
    repos_count = 0

    for tech_data in self.search_index.values():
        maturity_counts[tech_data['maturity_level']] += 1

        stars = tech_data['github_stars']
        if stars > 0:
            total_stars += stars
            repos_count += 1

```

```

        max_stars = max(max_stars, stars)

    stats['maturity_levels'] = dict(maturity_counts)
    stats['github_stats']['total_stars'] = total_stars
    stats['github_stats']['avg_stars'] = total_stars / repos_count if
repos_count > 0 else 0
    stats['github_stats']['max_stars'] = max_stars
    stats['github_stats']['technologies_with_repos'] = repos_count

    # Top tags
    tag_counts = defaultdict(int)
    for tech_data in self.search_index.values():
        for tag in tech_data['tags']:
            tag_counts[tag] += 1

    stats['top_tags'] = dict(sorted(tag_counts.items(), key=lambda x:
x[1], reverse=True)[:20])

    return stats

# API Interface for the search dictionary
class TechnologySearchAPI:
    """REST-like API interface for the technology search dictionary"""

    def __init__(self, search_dict: TechnologySearchDictionary):
        self.search_dict = search_dict

    def search_endpoint(self, query: str, search_type: str = "fuzzy",
                        filters: Optional[Dict] = None, sort_by: str =
"relevance",
                        limit: int = 20) -> Dict[str, Any]:
        """API endpoint for technology search"""
        try:
            # Convert string parameters to enums
            search_type_enum = SearchType(search_type.lower())
            sort_by_enum = SortBy(sort_by.lower())

            # Convert filters dict to SearchFilter object
            search_filter = None
            if filters:
                search_filter = SearchFilter(
                    categories=filters.get('categories'),
                    maturity_levels=filters.get('maturity_levels'),
                    license_types=filters.get('license_types'),
                    min_github_stars=filters.get('min_github_stars'),
                    max_complexity=filters.get('max_complexity'),
                    platforms=filters.get('platforms'),
                    programming_languages=filters.get('programming_languages')
                )

            # Perform search
            results = self.search_dict.search(query, search_type_enum,
            search_filter, sort_by_enum, limit)

```

```

# Convert results to API response format
api_results = []
for result in results:
    api_results.append({
        'id': result.technology_id,
        'name': result.name,
        'category': result.category,
        'description': result.description,
        'relevance_score': result.relevance_score,
        'match_type': result.match_type,
        'matched_fields': result.matched_fields,
        'tags': result.tags,
        'github_stars': result.github_stars,
        'maturity_level': result.maturity_level,
        'last_updated': result.last_updated
    })

return {
    'status': 'success',
    'query': query,
    'search_type': search_type,
    'total_results': len(api_results),
    'results': api_results,
    'timestamp': datetime.datetime.now().isoformat()
}

except Exception as e:
    return {
        'status': 'error',
        'message': str(e),
        'timestamp': datetime.datetime.now().isoformat()
    }

def suggestions_endpoint(self, partial_query: str, limit: int = 10) -
> Dict[str, Any]:
    """API endpoint for search suggestions"""
    try:
        suggestions = self.search_dict.get_suggestions(partial_query,
limit)
        return {
            'status': 'success',
            'partial_query': partial_query,
            'suggestions': suggestions,
            'timestamp': datetime.datetime.now().isoformat()
        }
    except Exception as e:
        return {
            'status': 'error',
            'message': str(e),
            'timestamp': datetime.datetime.now().isoformat()
        }

def stats_endpoint(self) -> Dict[str, Any]:

```

```

        """API endpoint for technology statistics"""
        try:
            stats = self.search_dict.get_technology_stats()
            return {
                'status': 'success',
                'statistics': stats,
                'timestamp': datetime.datetime.now().isoformat()
            }
        except Exception as e:
            return {
                'status': 'error',
                'message': str(e),
                'timestamp': datetime.datetime.now().isoformat()
            }

    def trending_endpoint(self, days: int = 30, limit: int = 10) ->
Dict[str, Any]:
        """API endpoint for trending technologies"""
        try:
            trending = self.search_dict.get_trending_technologies(days,
limit)

            return {
                'status': 'success',
                'trending_technologies': trending,
                'days': days,
                'timestamp': datetime.datetime.now().isoformat()
            }
        except Exception as e:
            return {
                'status': 'error',
                'message': str(e),
                'timestamp': datetime.datetime.now().isoformat()
            }

# Example usage and demonstration
def main():
    """Demonstrate the technology search dictionary"""

    print("=== Technology Search Dictionary Demonstration ===\n")

    # Initialize search dictionary
    search_dict = TechnologySearchDictionary()
    api = TechnologySearchAPI(search_dict)

    # Test different search types
    print("=== Fuzzy Search for 'AI' ===")
    results = search_dict.search("AI", SearchType.FUZZY, limit=5)
    for result in results:
        print(f"• {result.name} ({result.category})")
        print(f"  Relevance: {result.relevance_score:.1f}/100")
        print(f"  Match Type: {result.match_type}")
        print(f"  Description: {result.description[:100]}...")
        print()

```

```

# Test semantic search
print("=== Semantic Search for 'machine learning' ===")
results = search_dict.search("machine learning", SearchType.SEMANTIC,
limit=3)
for result in results:
    print(f"• {result.name}")
    print(f"  Matched Fields: {'', '.join(result.matched_fields)}")
    print(f"  Tags: {'', '.join(result.tags[:5])}")
    print()

# Test category search
print("=== Category Search for 'web' ===")
results = search_dict.search("web", SearchType.CATEGORY, limit=3)
for result in results:
    print(f"• {result.name} - {result.description[:80]}...")

# Test with filters
print("\n=== Filtered Search (High GitHub Stars) ===")
filters = SearchFilter(min_github_stars=50000)
results = search_dict.search("framework", SearchType.FUZZY,
filters=filters, limit=3)
for result in results:
    print(f"• {result.name}: {result.github_stars:,} stars")

# Get suggestions
print("\n=== Search Suggestions for 'py' ===")
suggestions = search_dict.get_suggestions("py", limit=5)
print(f"Suggestions: {'', '.join(suggestions)}")

# Get statistics
print("\n=== Technology Database Statistics ===")
stats = search_dict.get_technology_stats()
print(f"Total Technologies: {stats['total_technologies']}")
print(f"Categories: {len(stats['categories'])}")
print(f"Average GitHub Stars:
{stats['github_stats']['avg_stars']:.0f}")
print(f"Top Categories:")
for category, count in list(stats['categories'].items())[:5]:
    print(f"  {category}: {count}")

# Test API endpoints
print("\n=== API Endpoint Test ===")
api_response = api.search_endpoint("blockchain", "semantic", limit=2)
print(f"API Status: {api_response['status']}")
print(f"Results Found: {api_response['total_results']}")

print("\nTechnology Search Dictionary demonstration completed!")

if __name__ == "__main__":
    main()

```